

CareLite Healthcare Learning Platform

Low Level Design Document

Implementation design for repository structure, schemas, APIs, events, tenant routing, and DevOps layout.

Field	Value
Document Type	LLD
Version	0.1
Date	2026-05-15
Status	Draft for learning and implementation planning
Primary Stack	Java 21, Spring Boot 3.3.x, Spring Cloud 2023.0.x, Maven 15-module monorepo, Spring Cloud Gateway, OpenFeign, Amazon MSK/Kafka, MongoDB Atlas, AWS EKS, Cognito, S3, WAF, Shield, Jenkins, Argo CD

Document Control

Item	Details
Owner	CareLite learning project team
Audience	Developer, DevOps engineer, solution architect, product owner
Purpose	Guide a small healthcare platform that teaches production microservice patterns.
Scope Level	MVP plus platform learning patterns

1. Repository Structure

```
carelite-platform/  
  pom.xml  
  modules/  
    carelite-common/  
    carelite-security/  
    carelite-tenancy/  
    carelite-observability/  
    carelite-events/  
    carelite-test-support/  
    api-gateway/  
    identity-service/  
    tenant-service/  
    clinic-service/  
    billing-service/  
    document-service/  
    notification-service/  
    audit-event-service/  
    batch-service/  
    contract-tests/  
  deploy/  
    helm/carelite/  
    argocd/  
    jenkins/  
    ingress/  
  infra/  
    docker-compose.yml  
    observability/
```

1.1 Version and Dependency Baseline

Concern	Decision
Java	Java 21 with records for immutable DTOs and virtual threads for selected blocking service workloads.
Spring	Spring Boot 3.3.x and Spring Cloud 2023.0.x. Spring official compatibility maps 2023.0.x to Boot 3.3.x/3.2.x.
Build	Maven multi-module reactor with exactly 15 modules in the first implementation baseline.
Data	Amazon RDS PostgreSQL per-tenant schemas, MongoDB Atlas event store and CQRS read models, ElastiCache Redis, Amazon S3.
Messaging	Amazon MSK/Kafka with 48 partitions, Apache Avro, Confluent Schema Registry, Transactional Outbox.
Security	AWS Cognito OAuth2 with custom JWT claims, Spring Security RBAC, IAM/IRSA, WAF, Shield, Falco, Trivy.

Concern	Decision
Architecture patterns	Event Sourcing, CQRS, Saga orchestration, Outbox, Idempotency, multi-tenancy, DDD bounded contexts.
Observability	Prometheus, Grafana, Loki, Fluent Bit, OpenTelemetry, Tempo, Alertmanager, Grafana Faro.
Batch and documents	Spring Batch, iText 8, Apache POI, EventArchivalJob to Amazon S3 Avro archive.
Containers	Google Jib builds OCI images without requiring Docker-in-Docker in Jenkins.
Quality	SonarQube quality gate, ArchUnit architecture tests, Testcontainers integration tests, WireMock HTTP dependency tests.

2. Common Module Contracts

Class or Package	Purpose
TenantContextHolder	Stores current tenant_id for request lifecycle.
TenantHeaders	Constants: X-Tenant-ID, X-Correlation-ID, Idempotency-Key.
DomainEventEnvelope	Shared event schema used by outbox and Kafka.
ApiError	Standard error response with code, message, timestamp, correlationId.
IdempotencyFilter	Checks repeated POST requests before controller execution.
TenantGatewayFilter	Spring Cloud Gateway filter that resolves, validates, and propagates tenant context.
FeignClientConfig	Shared OpenFeign timeout, retry, tracing, and Resilience4j configuration.
CircuitBreakerNames	Common naming convention for service-to-service circuit breakers.
AvroEventSchema	Schema-registry-backed contract marker for Kafka domain events.
MongoEventStorePort	Hexagonal port for append-only MongoDB Atlas event storage and CQRS projections.

3. Database Design

3.1 Master Schema

```
CREATE TABLE master.tenants (
  tenant_id      VARCHAR(80) PRIMARY KEY,
  slug           VARCHAR(60) NOT NULL UNIQUE,
  clinic_name    VARCHAR(200) NOT NULL,
  schema_name    VARCHAR(80) NOT NULL UNIQUE,
  status         VARCHAR(30) NOT NULL,
  admin_email    VARCHAR(200) NOT NULL,
  cognito_group_prefix VARCHAR(120),
  s3_prefix       VARCHAR(300) NOT NULL,
  provisioning_step VARCHAR(80),
  idempotency_key VARCHAR(120) NOT NULL UNIQUE,
  created_at     TIMESTAMPTZ NOT NULL,
  updated_at     TIMESTAMPTZ NOT NULL
);
```

```
CREATE TABLE master.tenant_provisioning_steps (
  tenant_id      VARCHAR(80) NOT NULL,
  step_name      VARCHAR(80) NOT NULL,
  status         VARCHAR(30) NOT NULL,
```

```

    error_message TEXT,
    updated_at    TIMESTAMPTZ NOT NULL,
    PRIMARY KEY (tenant_id, step_name)
);

```

3.2 Tenant Schema Tables

```

CREATE TABLE patients (
  patient_id    UUID PRIMARY KEY,
  first_name    VARCHAR(100) NOT NULL,
  last_name     VARCHAR(100) NOT NULL,
  email         VARCHAR(200),
  mobile        VARCHAR(30),
  date_of_birth DATE,
  status        VARCHAR(30) NOT NULL,
  created_at    TIMESTAMPTZ NOT NULL,
  updated_at    TIMESTAMPTZ NOT NULL
);

CREATE TABLE doctors (
  doctor_id     UUID PRIMARY KEY,
  cognito_sub    VARCHAR(120),
  full_name      VARCHAR(160) NOT NULL,
  specialization VARCHAR(120),
  status        VARCHAR(30) NOT NULL
);

CREATE TABLE appointments (
  appointment_id UUID PRIMARY KEY,
  patient_id     UUID NOT NULL REFERENCES patients(patient_id),
  doctor_id      UUID NOT NULL REFERENCES doctors(doctor_id),
  starts_at      TIMESTAMPTZ NOT NULL,
  ends_at        TIMESTAMPTZ NOT NULL,
  status         VARCHAR(30) NOT NULL,
  reason         VARCHAR(300),
  idempotency_key VARCHAR(120)
);

```

3.3 Outbox and Consumer Idempotency

```

CREATE TABLE outbox_events (
  event_id      UUID PRIMARY KEY,
  event_type     VARCHAR(100) NOT NULL,
  aggregate_type VARCHAR(80) NOT NULL,
  aggregate_id   VARCHAR(120) NOT NULL,
  tenant_id      VARCHAR(80) NOT NULL,
  payload        JSONB NOT NULL,
  metadata       JSONB NOT NULL,
  status         VARCHAR(30) NOT NULL DEFAULT 'NEW',
  created_at     TIMESTAMPTZ NOT NULL,
  published_at   TIMESTAMPTZ
);

CREATE TABLE applied_events (
  event_id      UUID PRIMARY KEY,
  consumer_name VARCHAR(120) NOT NULL,
  is_replay     BOOLEAN NOT NULL DEFAULT FALSE,
  replay_job_id VARCHAR(120),
  applied_at    TIMESTAMPTZ NOT NULL
);

```

3.4 MongoDB Event Store

```

// Collection: event_store
{
  "_id": "event-uuid",
  "tenantId": "clinic_apollo",
  "eventType": "APPOINTMENT_BOOKED",
  "aggregateType": "APPOINTMENT",

```

```

    "aggregateId": "appointment-uuid",
    "sequenceNo": 12,
    "actorId": "cognito-sub",
    "actorRole": "RECEPTIONIST",
    "correlationId": "trace-id",
    "occurredAt": ISODate("2026-05-15T10:00:00Z"),
    "payload": {},
    "metadata": { "replay": false }
  }

Indexes:
  { tenantId: 1, occurredAt: -1 }
  { tenantId: 1, aggregateType: 1, aggregateId: 1, sequenceNo: 1 }
  { tenantId: 1, actorId: 1, occurredAt: -1 }
  { eventType: 1, occurredAt: -1 }

// Collection: replay_records
{ tenantId: 1, replayJobId: 1 }
{ tenantId: 1, status: 1, createdAt: -1 }

// CQRS read-model collections
appointment_timeline_by_patient:
{ tenantId: 1, patientId: 1, startsAt: -1 }
billing_summary_by_patient:
{ tenantId: 1, patientId: 1, invoiceDate: -1 }
audit_summary_by_actor:
{ tenantId: 1, actorId: 1, occurredAt: -1 }

```

4. Tenant Routing Implementation

- TenantGatewayFilter in api-gateway resolves tenant from subdomain, path override, or Cognito custom claim.
- Gateway validates tenant status through Redis cache and tenant-service fallback.
- Gateway writes X-Tenant-ID, X-Actor-ID, X-Roles, and X-Correlation-ID headers.
- A servlet filter in each downstream service reads X-Tenant-ID and validates it against a cached tenant registry.
- A transaction interceptor executes SET search_path TO <tenant_schema>, public before repository calls.
- TenantContextHolder is cleared in a finally block after each request.
- Async message consumers derive tenant_id from DomainEventEnvelope rather than HTTP headers.
- Admin cross-tenant operations must explicitly opt out of tenant schema routing.

5. Spring Cloud Gateway Design

```

spring:
  cloud:
    gateway:
      default-filters:
        - name: RequestRateLimiter
          args:
            redis-rate-limiter.replenishRate: 50
            redis-rate-limiter.burstCapacity: 100
            key-resolver: "#{@tenantAwareKeyResolver}"
      routes:
        - id: clinic-service
          uri: lb://clinic-service
          predicates:
            - Path=/api/v1/patients/**,/api/v1/appointments/**
          filters:
            - TenantGatewayFilter
            - name: CircuitBreaker
              args:
                name: clinicService

```

```
        fallbackUri: forward:/fallback/clinic
- id: billing-service
  uri: lb://billing-service
  predicates:
    - Path=/api/v1/invoices/**
  filters:
    - TenantGatewayFilter
    - name: RequestRateLimiter
    - name: CircuitBreaker
  args:
    name: billingService
    fallbackUri: forward:/fallback/billing
```

6. Feign and Circuit Breaker Design

Client	Caller	Purpose	Resilience Policy
TenantClient	api-gateway	Resolve tenant slug and status.	Short timeout, Redis cache fallback, circuit breaker.
ClinicClient	billing-service	Validate appointment or patient reference.	Timeout, no unsafe retry on POST, fallback to business error.
BillingClient	clinic-service	Optional invoice lookup for appointment detail.	Timeout, circuit breaker, cached read fallback where safe.
AuditClient	admin UI/backend	Query audit metadata if synchronous call is needed.	Timeout and graceful empty-state fallback.

```
resilience4j:
  circuitbreaker:
    instances:
      clinicService:
        slidingWindowSize: 20
        failureRateThreshold: 50
        waitDurationInOpenState: 30s
  timelimiter:
    instances:
      clinicService:
        timeoutDuration: 2s
feign:
  client:
    config:
      default:
        connectTimeout: 1000
        readTimeout: 2000
```

7. API Contracts

Service	Endpoint	Request Notes	Response
tenant-service	POST /api/v1/tenants/onboard	Idempotency-Key required.	tenant_id, status, admin user status.
clinic-service	POST /api/v1/patients	Tenant required; duplicate policy tenant-local.	patient_id and profile.
clinic-service	POST /api/v1/appointments	Idempotency-Key required; doctor slot lock.	appointment_id and status.
billing-service	GET /api/v1/invoices/{id}	Tenant required.	invoice detail.
audit-event-service	GET /api/v1/audit/events	Tenant filter from JWT.	paged audit events.

8. Domain Event Envelope

```
{
  "eventId": "uuid",
  "eventType": "APPOINTMENT_BOOKED",
  "aggregateType": "APPOINTMENT",
  "aggregateId": "appointment-uuid",
  "tenantId": "clinic_apollo",
  "actorId": "cognito-sub-or-system",
  "actorRole": "RECEPTIONIST",
  "correlationId": "trace-or-request-id",
  "occurredAt": "2026-05-15T10:00:00Z",
  "payload": {},
  "metadata": {
    "replay": false,
    "idempotencyKey": "abc-123"
  }
}
```

9. Kafka, Avro, and Schema Registry

Item	Low-Level Design
Cluster	Amazon MSK for managed Kafka in AWS.
Topic	domain.events with 48 partitions; key = tenant_id + '.' + aggregate_type + '.' + aggregate_id.
Serialization	Apache Avro for event payloads and metadata envelope.
Schema Registry	Confluent Schema Registry stores versioned event schemas with backward compatibility checks.
Producer guarantee	Transactional Outbox table plus relay prevents losing committed business events.
Consumer guarantee	At-least-once consumption plus applied_events table makes handlers idempotent.

10. Tenant Onboarding State Machine

Step	Action	Retry Behavior
VALIDATE	Check slug, email, idempotency.	Return existing tenant for same key.
MASTER_ROW	Insert master.tenants.	Upsert by idempotency key.
SCHEMA_CREATE	CREATE SCHEMA clinic_slug.	Ignore already exists if same tenant.
MIGRATE	Run Flyway for schema.	Resume from failed migration version.
S3_PREFIX	Create prefix marker.	Idempotent put object.
COGNITO	Create groups and first admin.	Check existing group/user first.
SEED	Insert default settings.	Upsert defaults.
ACTIVATE	Set tenant ACTIVE.	Safe to repeat.

11. AWS Integration Details

- Cognito: one user pool for the learning project; groups encode roles and tenant ownership.

- S3: document keys use tenants/{tenant_id}/patients/{patient_id}/filename.
- RDS: one PostgreSQL database with master schema and clinic_* schemas.
- MongoDB Atlas: the only event-store and CQRS read-model database in this architecture.
- Route 53: DNS records map tenant domains to CloudFront or ALB; health checks support failover routing.
- CloudFront, AWS WAF, and Shield: edge caching, rate-based rules, managed rules, and DDoS protection. WAF also attaches to ALB where needed.
- ACM: public certificates for CloudFront and ALB HTTPS listeners.
- ECR: one repository per service or one repository with service-name tags.
- EKS: each service has Deployment, Service, HPA, ConfigMap, and Secret references.
- Secrets Manager: DB credentials, Cognito app client settings, Kafka credentials, and MongoDB URI are injected at runtime through IRSA-compatible tooling.

12. Spring Batch and Document Jobs

Job	Implementation Notes
PatientBulkImportJob	Apache POI Excel reader, chunk size 500, idempotency key per row.
AppointmentReminderJob	Reads upcoming appointments, writes notification commands, suppresses duplicates.
InvoiceExportJob	Exports invoices to Excel using Apache POI and uploads to S3.
ReplayJob	Reads MongoDB event ranges and republishes safe events with replay=true metadata.
EventArchivalJob	Nightly job reads MongoDB event_store records and writes Avro files to S3 cold archive.
PDF generation	iText 8 renders prescriptions, invoices, and appointment summaries.

13. Kubernetes HA Baseline

Kubernetes Object	Baseline Requirement
Deployment	replicas >= 2, rolling update, resource requests/limits, topology spread constraints.
Service	ClusterIP for internal services; gateway exposed through ingress only.
Ingress	ALB ingress annotations for scheme, target type, certificate ARN, health checks, and WAF ACL ARN.
HPA	Scale on CPU/memory; add KEDA ScaledObject for Kafka consumers.
PDB	Prevent voluntary disruptions from evicting all replicas.
ServiceAccount	IRSA role per service for least-privilege access to S3, Secrets Manager, and other AWS APIs.

14. Jenkins Pipeline Design

```

stages:
  - checkout
  - mvn test
  - testcontainers integration tests
  - archunit architecture tests
  - wiremock contract and resilience tests

```


- sonarqube quality gate
- trivy fs scan
- google jib build service image
- trivy image scan
- push image to ECR
- update deploy/helm values image.tag
- commit and push manifest change

15. Helm and Argo CD Layout

```

deploy/
  helm/carelite/
    Chart.yaml
    values-dev.yaml
    templates/
      deployment.yaml
      service.yaml
      ingress.yaml
      hpa.yaml
      configmap.yaml
      serviceaccount.yaml
      pdb.yaml
      external-secret.yaml
  argocd/
    carelite-dev-application.yaml

```

16. Java Design Pattern Implementation Map

Pattern	Concrete Class Idea
Chain of Responsibility	GatewayFilter chain: CorrelationIdFilter -> TenantGatewayFilter -> RateLimitFilter -> RouteFilter.
Strategy	ProvisioningStep strategy implementations: SchemaStep, CognitoStep, S3Step, SeedDataStep.
Factory	DomainEventFactory creates typed event envelopes with tenant and correlation metadata.
Template Method	AbstractTenantProvisioner controls provision/validate/execute/mark-complete workflow.
Adapter	CognitoidentityAdapter, S3DocumentStorageAdapter, MongoEventStoreAdapter.
Repository	TenantRepository, PatientRepository, InvoiceRepository, EventStoreRepository.
Saga	TenantProvisioningSaga and AppointmentBillingSaga.
Builder	Immutable DTO and event envelope builders for readable test setup.

17. Testing Strategy

Test Type	Scope
Unit	Service methods, validators, ID and tenant helpers.
Slice	Controller tests with mocked service dependencies.
Integration	PostgreSQL Testcontainers for tenant schema routing and outbox.
Integration	MongoDB Testcontainers for event store append and audit queries.
Resilience	WireMock or MockWebServer tests for Feign timeout, fallback, and circuit breaker behavior.
Contract	Gateway to service API contracts.

Test Type	Scope
E2E smoke	Onboard tenant, login, create patient, book appointment, view audit.
DevOps	Jenkins dry run and Argo CD sync to dev namespace.

18. Initial Package Naming

Module	Base Package
carelite-common	com.carelite.common
carelite-security	com.carelite.security
carelite-tenancy	com.carelite.tenancy
carelite-observability	com.carelite.observability
carelite-events	com.carelite.events
carelite-test-support	com.carelite.testsupport
api-gateway	com.carelite.gateway
identity-service	com.carelite.identity
tenant-service	com.carelite.tenant
clinic-service	com.carelite.clinic
billing-service	com.carelite.billing
document-service	com.carelite.document
notification-service	com.carelite.notification
audit-event-service	com.carelite.audit
batch-service	com.carelite.batch
contract-tests	com.carelite.contracts